Concept 1: API Resource vs Database Entity

Let's start with what a **database entity** is. You already know this from your modeling work — it's a table in your database. `rfq`, `rfq_stage`, `rfq_supplier`, `rfq_line_item` — those are all entities. Each one has attributes (columns) and relationships (FK/PK links).

Now, an **API resource** is something different. It's what the outside world (your front-end) is allowed to see and interact with. It's a door you deliberately open for the client to use.

Here's the key: **not every table gets a door.**

Think about it with your RFQ example. When a user clicks "Create New RFQ" on the front-end, they fill in a form and hit submit. That's **one** API call: `POST /rfq-manager/v1/rfqs`. But what happens inside your service when it receives that call? It might create a row in the `rfq` table, then automatically create a row in `rfq_stage` (the initial stage), maybe initialize some other records. The user didn't ask for any of that — your service just knows to do it internally.

So `rfq` is both an entity AND an API resource (because the front-end interacts with it directly). But `rfq_stage` might only be an entity — it gets created behind the scenes, the front-end never calls `POST /rfq-manager/v1/rfq-stages` to create one manually.

That's what your bosses meant by "intermediate entities built on demand."

The question you should always ask: "Does the user on the front-end need to directly create, read, update, or delete this thing?" If yes → it's an API resource. If no → it's just an internal entity that your service manages on its own.

Part 1 — Concepts You Must Learn (Phase A)

Each concept below is explained simply, then linked to what was said in the meeting. **Read these in order — each builds on the previous.**

1. API Resource vs Database Entity

	API Resource	Database Entity
What it is	What the **client** (front-end) sees and interacts with	A **table** in your database
Who knows about it	External consumers (UI, chatbot, other services)	Only the back-end code internally
Example	`POST /rfqs` → "Create an RFQ"	Behind the scenes: inserts into `rfq`, `rfq_stage` (initial), maybe `rfq_reminder`, etc.

> **Meeting link:** "il y a des entités qui sont intermédiaires qui n'exposent pas de CRUD — elles sont construites à la demande." When you create an RFQ, `rfq_stage` rows are auto-created. The client never calls `POST /rfq_stages` — they're **intermediate entities**.

Key insight: Not every table becomes an endpoint. Your API is a **simplified, business-oriented view** of your data model.

Most of the time, an API resource **maps to** a database entity. Like `rfq` is a table in your database AND an API resource because you expose endpoints for it.

But your bosses said something specific: "API resources can reflect a data model but **don't have to**." This means sometimes an API resource **doesn't** have its own table. It could be a combination of data from multiple tables, or a computed result, or a virtual concept that the front-end needs but that doesn't live as a single table in the database.

So the real picture is:

- **Database entity** = a table in your database. Always.
- **API resource** = something you expose to the front-end. Usually maps to an entity, but not necessarily.

- **Intermediate entity** = a database table that is never exposed. Internal only.

1) Business object (concept)

What humans talk about: “RFQ”, “Stage”, “Reminder”, “File”.

2) API Resource (public object)

A business object that is exposed through endpoints.

Example: RFQ resource → `/rfqs`.

3) Database entity / table (storage object)

How we store data.

Often a resource maps to a table, but not always.

4) Supporting table (internal helper)

A table that exists to support a resource but is not necessarily exposed directly as a resource.

Example: `stage_template` supports “workflow”; it may be read-only in V1.

Rule you can memorize:

- If it needs **its own lifecycle in UI** (create/list/update/delete independently) → it becomes an **API resource**.
- If it exists mainly to support another object and UI doesn’t manage it directly → it’s a **supporting table**.